how i built a *transformer to add 2 two-digit numbers on pen and paper.

FORGEFORMER

aritro shome

tldr: demo it at https://aritro.is-a.dev/forgeformer and read the blog at https://silicognition.is-a.dev/post2.html

* i do not claim to adopt all components (Layer Norm, FFNs, etc.) of the transformer religiously. i have modified certain things to make my objective of this exercise easier. be patient, not pedantic.

WHY ON EARTH WOULD I DO THIS?

keywords: mechanistic interpretability, QK and OV circuits, how attention layers specialize in tasks.

WHY ON EARTH WOULD I DO THIS?

1. having recently worked with a professor on *saliency maps* for explainable vision transformers, i naturally dug deeper into explainability and interpretability and discovered mechanistic interpretability (thank you Neel Nanda for this amazing topic)
2. aptly called the neurosurgery for models, mech-int goes deep into the models, and translates the numerous linear algebra operations (especially those of the attention heads) into human-understandable algorithms. this is useful for many reasons.
3. an academic challenge never bored me, plus i was intrigued to do this task for a long time. free will, if you will (pun intended)

ARCHITECTURE OF FORGEFORMER

keywords: attention mechanism, key-query-value matrices, softmax, embeddings, tokens

ARCHITECTURE OF FORGEFORMER – PART 1

1. *d_model* has to be as small as possible, something like 512 is going to be a nightmare (read: impossible task) to solve on paper. I went with *d_model* = 3, one dimension dedicated for position, one dimension dedicated for digit value, and one for carry related purposes.
2. The input token length is fixed at 5: a b c d <EOS>. the addition to be performed is ab + cd. a, b, c, d are all valid digits.
3. layer norms, FFNs and embedding and unembedding matrices would only increase the number of parameters i have to figure out, which would deviate from my task of analyzing attention heads and their relation to the task at hand. hence little "hacks" have been used there :P

ARCHITECTURE OF FORGEFORMER – PART 2

1. there are 2 attention layers. one for units digit, one for tens digit. although this type of predefinition is counter intuitive to goal of mechint (in mechint we go from model weights → human algorithms not the other way around), this exercise is my way of sort of coming up with the trivial-est of examples to figure out how QK and OV circuits define the tasks and outputs.

The red arrow here signifies residual connection.

FORGEFORMER ATTENTION LAYER 1

keywords: attention mechanism, how key and query “attend” to required positions

FORGEFORMER ATTENTION LAYER 1 – PART 1

1. to get the units sum the EOS token needs to find only the digits at the unit positions (indices 2 and 4 in the list of tokens, considering one-indexing).
2. think of the EOS token as a Man on a Hill shouting through a megaphone: i am looking for anyone standing in a position indicated by a negative number!".
3. The Query matrix is the man shouting, and the Key matrix returns the positions of where affirmative echoes should come (very weak example)

Red border => weights I derived by hand

$$\boxed{W_q^1 = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 10 \end{pmatrix}} \qquad \boxed{W_{k1} = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & -10 \end{pmatrix}}$$

$$X_1 = \begin{pmatrix} 0 & 1 & 1 \\ 0 & 7 & -1 \\ 0 & 2 & 1 \\ 0 & 5 & -1 \\ 0 & 0 & 1 \end{pmatrix} \quad K_1 = \begin{pmatrix} 0 & 0 & -10 \\ 0 & 0 & 10 \\ 0 & 0 & -10 \\ 0 & 0 & 10 \\ 0 & 0 & -10 \end{pmatrix} \quad Q_1 = \begin{pmatrix} 0 & 0 & 10 \\ 0 & 0 & -10 \\ 0 & 0 & 10 \\ 0 & 0 & -10 \\ 0 & 0 & 10 \end{pmatrix}$$

$$KQ_1 = \begin{pmatrix} -100 & 100 & -100 & 100 & -100 \\ 100 & -100 & 100 & -100 & 100 \\ -100 & 100 & -100 & 100 & -100 \\ 100 & -100 & 100 & -100 & 100 \\ -100 & \underline{100} & -100 & \underline{100} & -100 \end{pmatrix}$$

FORGEFORMER ATTENTION LAYER 1 – PART 2

1. once the K-Q handshake tells us where to look, the V will tell us how to pluck and use those values in those positions.
2. Analogy: K-Q is the librarian, you ask them where is a book on reptiles, they show you the shelf. Now you as the reader (V matrix) will actually use the contents in the book in those shelf.

$$\boxed{W_{v1} = \begin{pmatrix} 0 & 0 & 0 \\ 2.0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}}$$

$$KQ_1 = \begin{pmatrix} -100 & 100 & -100 & 100 & -100 \\ 100 & -100 & 100 & -100 & 100 \\ -100 & 100 & -100 & 100 & -100 \\ 100 & -100 & 100 & -100 & 100 \\ -100 & 100 & -100 & 100 & -100 \end{pmatrix}$$

why 2?

$$v_1 = \begin{pmatrix} 2 & 0 & 0 \\ 14 & 0 & 0 \\ 4 & 0 & 0 \\ 10 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}$$

$$\text{attn}_1 = \begin{pmatrix} 0 & 0.5 & 0 & 0.5 & 0 \\ 0.33 & 0 & 0.33 & 0 & 0.33 \\ 0 & 0.5 & 0 & 0.5 & 0 \\ 0.33 & 0 & 0.33 & 0 & 0.33 \\ 0 & \underline{0.5} & 0 & \underline{0.5} & 0 \end{pmatrix}$$

FORGEFORMER ATTENTION LAYER 1 — PART 3

$$x_1 = \begin{pmatrix} 0 & 1 & 1 \\ 0 & 7 & -1 \\ 0 & 2 & 1 \\ 0 & 5 & -1 \\ 0 & 0 & 1 \end{pmatrix}, \quad W_{k1} = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & -10 \end{pmatrix}, \quad W_{v1} = \begin{pmatrix} 0 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}, \quad W_{q1} = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 10 \end{pmatrix}$$

$$k_1 = x_1 W_{k1} = \begin{pmatrix} 0 & 0 & -10 \\ 0 & 0 & 10 \\ 0 & 0 & -10 \\ 0 & 0 & 10 \\ 0 & 0 & -10 \end{pmatrix}, \quad q_1 = x_1 W_{q1} = \begin{pmatrix} 0 & 0 & 10 \\ 0 & 0 & -10 \\ 0 & 0 & 10 \\ 0 & 0 & -10 \\ 0 & 0 & 10 \end{pmatrix}, \quad v_1 = x_1 W_{v1} = \begin{pmatrix} 1 & 0 & 0 \\ 7 & 0 & 0 \\ 2 & 0 & 0 \\ 5 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}$$

$$k_1 q_1^T = \begin{pmatrix} -100 & 100 & -100 & 100 & -100 \\ 100 & -100 & 100 & -100 & 100 \\ -100 & 100 & -100 & 100 & -100 \\ 100 & -100 & 100 & -100 & 100 \\ -100 & 100 & -100 & 100 & -100 \end{pmatrix} \qquad \text{attn}_1 = \text{Softmax}(k_1 q_1^T) = \begin{pmatrix} 0 & 0.5 & 0 & 0.5 & 0 \\ 0.33 & 0 & 0.33 & 0 & 0.33 \\ 0 & 0.5 & 0 & 0.5 & 0 \\ 0.33 & 0 & 0.33 & 0 & 0.33 \\ 0 & 0.5 & 0 & 0.5 & 0 \end{pmatrix}$$

$$\text{Output}_1 = \text{attn}_1 \times v_1 = \begin{pmatrix} 6 & 0 & 0 \\ 1 & 0 & 0 \\ 6 & 0 & 0 \\ 1 & 0 & 0 \\ 6 & 0 & 0 \end{pmatrix}$$

$$x_2 = x_1 + \text{Output}_1 = \begin{pmatrix} 6 & 1 & 1 \\ 1 & 7 & -1 \\ 6 & 2 & 1 \\ 1 & 5 & -1 \\ 6 & 0 & 1 \end{pmatrix}$$

FORGEFORMER ATTENTION LAYER 2

keywords: attention mechanism, how key and query “attend” to required positions, how residual connection information of previous layer(s) is used

FORGEFORMER ATTENTION LAYER 2 – PART 1

Same intuitions as before.

Last time we focused on positions 2 and 4, this time 1 and 3 for the tens digit.

$$x_2 = \begin{pmatrix} 6 & 1 & 1 \\ 1 & 7 & -1 \\ 6 & 2 & 1 \\ 1 & 5 & -1 \\ 6 & 0 & 1 \end{pmatrix}, \quad \boxed{W_{q2} = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 10 \end{pmatrix}, \quad W_{k2} = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 10 \end{pmatrix}}$$

If you recall, in Wk1 we had -10. here +10. why?

$$k_2 = x_2 W_{k2} = \begin{pmatrix} 0 & 0 & 10 \\ 0 & 0 & -10 \\ 0 & 0 & 10 \\ 0 & 0 & -10 \\ 0 & 0 & 10 \end{pmatrix}, \quad q_2 = x_2 W_{q2} = \begin{pmatrix} 0 & 0 & 10 \\ 0 & 0 & -10 \\ 0 & 0 & 10 \\ 0 & 0 & -10 \\ 0 & 0 & 10 \end{pmatrix}$$

$$kq_2 = \begin{pmatrix} 100 & -100 & 100 & -100 & 100 \\ -100 & 100 & -100 & 100 & -100 \\ 100 & -100 & 100 & -100 & 100 \\ -100 & 100 & -100 & 100 & -100 \\ \underline{100} & -100 & \underline{100} & -100 & 100 \end{pmatrix}$$

See how signs are opposite of kq1

FORGEFORMER ATTENTION LAYER 1 – PART 2

Intuitions same as before.

This time we shall change for the tens digit.

$$\boxed{W_{v2} = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 3.0 & 0 \\ 0 & 0 & 0 \end{pmatrix}}$$

Why 3?

$$v_2 = x_2 W_{v2} = \begin{pmatrix} 0 & 1 \times 3 & 0 \\ 0 & 7 \times 3 & 0 \\ 0 & 2 \times 3 & 0 \\ 0 & 5 \times 3 & 0 \\ 0 & 0 \times 3 & 0 \end{pmatrix} = \begin{pmatrix} 0 & 3 & 0 \\ 0 & 21 & 0 \\ 0 & 6 & 0 \\ 0 & 15 & 0 \\ 0 & 0 & 0 \end{pmatrix}$$

$$\text{attn}_2 = \text{Softmax}(k_2 q_2^T) = \begin{pmatrix} 0.33 & 0 & 0.33 & 0 & 0.33 \\ 0 & 0.5 & 0 & 0.5 & 0 \\ 0.33 & 0 & 0.33 & 0 & 0.33 \\ 0 & 0.5 & 0 & 0.5 & 0 \\ 0.33 & 0 & 0.33 & 0 & 0.33 \end{pmatrix}$$

$$\text{Output} = \begin{pmatrix} 12 & 4 & 1 \\ 2 & 25 & -1 \\ 12 & 5 & 1 \\ 2 & 23 & -1 \\ \mathbf{12} & \mathbf{3} & \mathbf{1} \end{pmatrix}$$

FORGEFORMER ATTENTION LAYER 1 — PART 3

$$x_2 = \begin{pmatrix} 6 & 1 & 1 \\ 1 & 7 & -1 \\ 6 & 2 & 1 \\ 1 & 5 & -1 \\ 6 & 0 & 1 \end{pmatrix}, \quad W_{q2} = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 10 \end{pmatrix}, \quad W_{k2} = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 10 \end{pmatrix} \quad k_2 = x_2 W_{k2} = \begin{pmatrix} 0 & 0 & 10 \\ 0 & 0 & -10 \\ 0 & 0 & 10 \\ 0 & 0 & -10 \\ 0 & 0 & 10 \end{pmatrix}, \quad q_2 = x_2 W_{q2} = \begin{pmatrix} 0 & 0 & 10 \\ 0 & 0 & -10 \\ 0 & 0 & 10 \\ 0 & 0 & -10 \\ 0 & 0 & 10 \end{pmatrix}$$

$$\text{attn}_2 = \text{Softmax}(k_2 q_2^T) = \begin{pmatrix} 0.33 & 0 & 0.33 & 0 & 0.33 \\ 0 & 0.5 & 0 & 0.5 & 0 \\ 0.33 & 0 & 0.33 & 0 & 0.33 \\ 0 & 0.5 & 0 & 0.5 & 0 \\ 0.33 & 0 & 0.33 & 0 & 0.33 \end{pmatrix} \quad W_{v2} = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 3.0 & 0 \\ 0 & 0 & 0 \end{pmatrix} \quad v_2 = x_2 W_{v2} = \begin{pmatrix} 0 & 1 \times 3 & 0 \\ 0 & 7 \times 3 & 0 \\ 0 & 2 \times 3 & 0 \\ 0 & 5 \times 3 & 0 \\ 0 & 0 \times 3 & 0 \end{pmatrix} = \begin{pmatrix} 0 & 3 & 0 \\ 0 & 21 & 0 \\ 0 & 6 & 0 \\ 0 & 15 & 0 \\ 0 & 0 & 0 \end{pmatrix}$$

$$\text{Output} = \begin{pmatrix} 12 & 4 & 1 \\ 2 & 25 & -1 \\ 12 & 5 & 1 \\ 2 & 23 & -1 \\ \mathbf{12} & \mathbf{3} & \mathbf{1} \end{pmatrix}$$

GETTING THE SUM OUT OF THE FORGEFORMER

keywords: how not to do unembedding by cheating a little bit (using the plus operator)

GETTING THE SUM OUT OF THE FORGEFORMER

in a traditional transformer, we have unembedding matrix which converts the final logits to a corresponding token. However, our goal is to study attention layers and how they perform tasks, so we modify the final layer a little bit.

```
units_sum = x[-1, 0]
    tens_sum = x[-1, 1]
    carry = 0
    if units_sum >= 10:
        carry = 1
        units_sum -= 10
    return (int((tens_sum + carry) * 10 +
units_sum), states_map)
```

I grab the EOS row (last row of the residual stream output after 2nd layer. The first dimension has the carry and sum of unit's digit and the second dimension has the sum of tens' digit. due to this Implementation of extracting the sum, the Forgeformer actually "cheats" into overflowing the 2nd dimension beyond 9 to handle sums > 99 as well.

TAKEAWAY

keywords: not The Chainsmokers song, what i learnt and hopefully taught

TAKEAWAY

1. Properly implementing the hundredth carry, right now it cheats.
2. Adding FFNs and Layer Norms (if needed to see what they contribute intuitively)
3. Generalizing an approach for n digit + m digit, n != m (using computer trained models of course)
4. Carrying our concepts about what the attention heads actually do and checking with open-source models
5. Training one model for the same task as Forgeformer using Pytorch and comparing their weights with mine.
6. Doing all this for other trivial task

THANK YOU

Twitter: https://x.com/silicognition
Personal site: https://aritro.is-a.dev/

Feel free to reach out (even if you're pedantic)